

Dual-mode: hardware **mode bit** distinguishes **kernel mode** (full access; runs OS) from **user mode** (restricted; runs user programs). **Privileged instructions** (I/O, halt, set timer, modify page table) trap if executed in user mode. **Trap:** synchronous, software-initiated transfer to kernel (`syscall`, breakpoint). **Interrupt:** asynchronous, hardware-initiated (timer, I/O device). **Exception:** asynchronous fault (divide-by-zero, page fault, invalid opcode).

System call: user loads `syscall` number + args into registers, executes trap; CPU switches to kernel mode, dispatches via `syscall` table, runs handler, returns to user mode with result.

1 Introduction

Abstraction: hide hardware variation behind a uniform interface (process, file, virtual memory) for portability and programmability. **Mechanism:** how a capability is provided (e.g., context-switch routine, `syscall` dispatcher). **Policy:** which decision a mechanism makes (e.g., which page to evict, which process to run). Separating mechanism from policy keeps the mechanism **policy-agnostic**, enabling policy change without rewriting mechanism.

Kernel architectures

- Monolithic:** all OS services share one kernel address space. Fast (no IPC), well-understood, but tightly coupled. [Linux](#), [Windows NT](#), [most Unix](#).
- Microkernel:** tiny kernel exposes only IPC, scheduling, address-space mgmt; other services run as user-mode servers. Modular and robust, but slower from IPC overhead.
- Hybrid / modular:** monolithic core with loadable kernel modules at runtime ([Linux LKM](#), [macOS XNU](#) = Mach + BSD).

Virtualisation

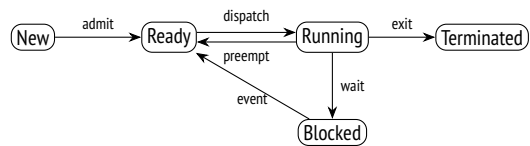
Type 1 hypervisor runs on bare metal ([ESXi](#)); **Type 2** runs as an app in a host OS ([VirtualBox](#), [QEMU](#)).

2 Processes

A **process** is a running instance of a **program** (static, on disk). It bundles three contexts:

- Memory:** **text** (code, RO), **data** (globals/statics), **heap** (grows up), **stack** (grows down).
- Hardware:** GPRs, **PC**, **SP**, status flags.
- OS (PCB):** PID, PPID, state, saved regs+PC, MMU info, FD table, scheduling info, signal handlers, accounting.

2.1 Process states



Unix variant adds **Stopped** (suspended via `SIGSTOP`; resumed via `SIGCONT`). **Zombie:** exited, awaiting reap (PCB held for exit status); leaks PIDs if unreaped. **Orphan:** parent died first; reparented to `init` (PID 1).

2.2 Context switch

Save running process's CPU registers + PC + MMU state into its PCB; restore next process's. Triggered by **timer interrupt**, blocking I/O, preemptioin, or voluntary yield. **Overhead:** register save/restore, TLB flush, cache pollution.

2.3 Unix process API

- `pid_t fork(void)`: clone caller. Returns child PID to parent, 0 to child, `-1` on error.

- `int exec*(mode, path, argv[], envp[])`: replace image with new program. Same PID; FDs inherited (unless `FD_CLOEXEC`); signal handlers reset. Returns only on failure.
- `pid_t wait(&status)`: block until *any* child terminates; reap zombie; return its PID.
- `pid_t waitpid(pid, &status, opts)`: wait for specific child; `WNOHANG` for non-blocking.
- `void exit(status)`: terminate; flush buffers; signal parent. Lower 8 bits of `status` returned.
- `pid_t getpid()/getppid()`: own / parent PID.

Status macros

(interpret `status` from `wait`):

- `WIFEXITED(s)`: true if child exited normally.
- `WEXITSTATUS(s)`: exit code (use only if `WIFEXITED`).
- `WIFSIGNALED(s)`: terminated by signal.
- `WTERMSIG(s)`: terminating signal number.

Copy-on-write (COW) and FD inheritance

Copy-on-write: `fork` initially shares parent's pages read-only; on first write, kernel page-faults and copies. Avoids full memory copy. **FD inheritance:** child inherits parent's open FDs, sharing the same file-offset entry; both see seeks and writes.

fork() process tree

$n \text{ fork}() \Rightarrow 2^n$ procs; `printf` runs 2^k times after k forks. `fork() && fork()  $\Rightarrow$  3` (child short-circuits on 0).

Canonical fork-exec-wait

```
pid_t pid = fork();
if (pid == -1) { /* error */ }
else if (pid == 0) { /* child */
    execv("/bin/prog", argv);
    _exit(1); /* exec failed */
} else { /* parent */
    int s; waitpid(pid, &s, 0);
    if (WIFEXITED(s))
        /* WEXITSTATUS(s) */;
}
```

exec* family

- `l` = list args (variadic, NULL-terminated); `v` = vector `argv[]`.
- `p` = search PATH on bare names; `e` = pass custom `envp[]`.
- Combinations: `exec1`, `execv`, `exec1p`, `execvp`, `execle`, `execve`. Only `execve` is the actual `syscall`; others are libc wrappers.

3 Scheduling

The **scheduler** chooses which Ready process runs next.

Criteria

- Turnaround** TAT = finish — arrival.
- Waiting** WT = time in Ready queue.
- Response:** time to first response.
- Throughput:** tasks per unit time.
- CPU utilisation:** useful CPU time / total time.
- Fairness:** every process eventually runs (no starvation).

Averages: $\overline{WT} = \frac{1}{n} \sum WT_i$, $\overline{TAT} = \frac{1}{n} \sum TAT_i$.

Preemptive vs non-preemptive

Non-preemptive: process holds CPU until it blocks or yields. **Pre-emptive:** timer interrupt forcibly suspends after a quantum.

3.1 Algorithms

FCFS

Non-preemptive FIFO by arrival.

- `int exec*(mode, path, argv[], envp[])`: replace image with new program. Same PID; FDs inherited (unless `FD_CLOEXEC`); signal handlers reset. Returns only on failure.
- Convoy effect:** short I/O-bound jobs queue behind a long CPU-bound job; CPU runs long job $\rightarrow$ I/O idle, then short jobs do quick I/O $\rightarrow$ CPU idle.

SJF

Non-preemptive; pick smallest expected total CPU burst.

- Optimal** mean waiting time when all jobs are available at $t = 0$ (otherwise SRTF wins).
- Needs burst prediction; **starvation** of long jobs.

SRTF

Preemptive SJF: a newly arrived job with shorter remaining time preempts the running one.

Burst prediction (exponential average)

$$\tau_{n+1} = \alpha \tau_n + (1 - \alpha) \tau_n, \quad 0 \leq \alpha \leq 1.$$
Larger α weights recent bursts; smaller weights history.

Round Robin (RR)

Preemptive FIFO with quantum q .

- Fair; no starvation.
- Response bound:** $\leq (n - 1)q$ for n ready tasks.
- Quantum trade-off:** small $q \rightarrow$ context-switch overhead; large $q \rightarrow$ FCFS-like.

Priority

Pick highest priority; preemptive or non-preemptive variant.

- Risk: **starvation** of low-priority tasks.
- Mitigation: **aging** — gradually raise priority of waiting tasks.

MLFQ

Multiple priority queues; learn behaviour adaptively.

- Higher-priority queue runs first.
- RR within a level.
- New jobs enter at top; **demote** a job that uses its full quantum.
- A job that yields (I/O) before quantum end keeps its priority.
- Periodic **boost** — push all jobs to top — prevents starvation.

Effect: I/O-bound stays high (good response); CPU-bound sinks low with longer quanta (good throughput).

I/O-bound vs CPU-bound

CPU-bound: long bursts of computation, rare I/O; favours longer quanta, optimise turnaround. **I/O-bound:** short CPU bursts between I/O waits; favours short quanta and high priority, optimise response.

Environments

Batch: no user; favour throughput, non-preemptive ok. **Interactive:** human in loop; preemptive, optimise response time and predictability. **Real-time:** *hard* (deadline must hold; aircraft, control) vs *soft* (best-effort; multimedia); deadline-aware schedulers.

Worked example

Three jobs arriving at $t = 0$ with bursts $P_1 = 24$, $P_2 = 3$, $P_3 = 3$.

- FCFS** order $P_1 \rightarrow P_2 \rightarrow P_3$: waits 0, 24, 27; $\overline{WT} = 17$.
- SJF** order $P_2 \rightarrow P_3 \rightarrow P_1$: waits 0, 3, 6; $\overline{WT} = 3$.
- RR**, $q = 4$: waits 6, 4, 7; $\overline{WT} = 5.6$.

Burst-prediction example

$\alpha = \frac{1}{2}$, $\tau_0 = 10$, observed $t_0 = 6$: $\tau_1 = \frac{1}{2}(6) + \frac{1}{2}(10) = 8$. With $t_1 = 4$: $\tau_2 = \frac{1}{2}(4) + \frac{1}{2}(8) = 6$.

Scheduler classes (Linux)

SCHED_OTHER: default time-sharing (CFS). **SCHED_FIFO:** real-time, no quantum — runs until block or yield. **SCHED_RR:** real-time round-

robin within a priority. **SCHED_IDLE:** lowest priority, runs only when idle.

4 Inter-Process Communication

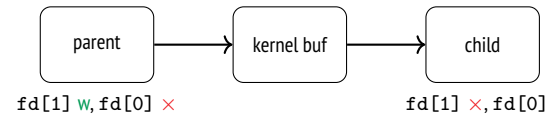
Independent address spaces require explicit **IPC** for cooperation.

Shared memory vs message passing

- Shared memory:** fast (direct memory access after setup); caller must synchronise; not distributed-friendly.
- Message passing:** slower (one `syscall` per send/receive); synchronisation implicit if blocking; portable to distributed systems.

4.1 Pipes

`int pipe(int fd[2])` creates a half-duplex byte stream: `fd[0]` read end, `fd[1]` write end; circular kernel buffer with implicit producer/consumer sync.



- Pipe *before* `fork`; close unused ends in each process (needed for EOF).
- `dup2(fd, STDIN/OUT_FILENO)` redirects standard streams to/from pipe.
- Writing to pipe with closed read end raises `SIGPIPE`; reader sees EOF when all write ends close.

Pipe+fork skeleton:

```
int fd[2]; pipe(fd);
if (fork() == 0) { /* child reads */
    close(fd[1]);
    read(fd[0], buf, n);
    close(fd[0]);
} else { /* parent writes */
    close(fd[0]);
    write(fd[1], msg, len);
    close(fd[1]);
    wait(NULL);
}
```

4.2 Named pipes (FIFO)

`int mkfifo(path, mode)` creates a persistent filesystem entry. Unrelated processes can open it; otherwise behaves like a pipe.

	Pipe	FIFO
related procs only	yes	no
persistent	no	yes (in FS)
duplex	half	half

4.3 Shared memory (POSIX)

- shm_open**(name, oflag, mode) creates/opens an SHM object.
- ftruncate**(fd, size) sets size.
- mmap**(addr, len, prot, flags, fd, off) maps it into the process address space.
- munmap**, **shm_unlink** to release.

Skeleton:

```
int fd = shm_open("/buf",
                  O_CREAT|O_RDWR, 0600);
ftruncate(fd, N);
void *p = mmap(NULL, N,
               PROT_READ|PROT_WRITE,
               MAP_SHARED, fd, 0);
/* use *p; sync via sem/mutex */
munmap(p, N);
shm_unlink("/buf");
```

4.4 Message passing

Primitives `send(dest, msg)`, `receive(src, msg)` via syscalls.

- **Direct** naming vs **indirect** via **mailbox/port** (multiple peers).
- **Synchronous** (blocking, **rendezvous**) vs **asynchronous** (non-blocking; needs buffer).
- **Capacity**: zero (rendezvous), bounded (sender blocks on full), unbounded (never blocks).

4.5 Signals

Asynchronous notification to a process.

- **kill**(pid, sig): send signal.
- **signal**(sig, handler) or **sigaction**(...): register handler. Special handlers: `SIG_DFL`, `SIG_IGN`.
- **Common signals**: `SIGINT` (2, ^C, terminate); `SIGTERM` (15, polite kill, catchable); `SIGKILL` (9, force kill, uncatchable); `SIGSTOP/SIGCONT` (pause/resume; `SIGSTOP` uncatchable); `SIGCHLD` (child state change); `SIGSEGV` (invalid mem access; default core dump); `SIGPIPE` (write to pipe with closed read end).
- **Default actions**: terminate, terminate+core, ignore, stop, continue.
- **Signal-safety**: only async-signal-safe calls (e.g., `write`, `_exit`) inside handlers; not `printf/malloc`. Idiom: set volatile `sig_atomic_t` flag in handler, handle in main loop.
- `sigaction` extends `signal`: a `sigset_t` mask blocks chosen signals during the handler; `SA_RESTART` auto-restarts interrupted slow syscalls; portable across Unix.

Reaping children with SIGCHLD

Avoid zombies asynchronously by handling `SIGCHLD`:

```
void reap(int s) {
    while (waitpid(-1, NULL, WNOHANG) > 0);
}
sigaction(SIGCHLD, &(struct sigaction){
    .sa_handler = reap,
    .sa_flags   = SA_RESTART|SA_NOCLDSTOP
}, NULL);
```

Loop with `WNOHANG` because `SIGCHLD` is not queued – multiple simultaneous child exits coalesce into one signal.

5 Threads

A **thread** is a lightweight execution context within a process: a separate flow of control sharing the process's address space.

Shared vs per-thread

- **Shared**: text, data, heap, FDs, signal handlers.
- **Per-thread**: TID, registers, PC, stack, `errno`, thread-local storage.

Thread context switch saves only registers + SP (no MMU/page-table change) – much cheaper than process switch.

5.1 Threading atomics

- **User-level (M:1)**: managed by user library; kernel sees one thread. Cheap; **any** blocking syscall blocks the whole process; cannot exploit multicore.
- **Kernel-level (1:1)**: each user thread maps to a kernel thread. True parallelism; non-blocking I/O; thread ops are syscalls (slower). **Linux NPTL**.
- **Hybrid (M:N)**: many user threads multiplexed over fewer kernel threads. Flexible scheduling and multicore use; complex.

5.2 pthreads core API

- `int pthread_create(&tid, attr, start, arg)`: spawn thread running `start(arg)`; `attr=NULL` for defaults.
- `int pthread_join(tid, &status)`: block until `tid` exits; receive return value.
- `void pthread_exit(retval)`: terminate calling thread; implicit on return.
- `pthread_t pthread_self()`: own TID.

Advantages: cheap creation/switch; direct memory sharing (no IPC); responsive; multicore parallelism.

Disadvantages: shared data needs synchronisation; one-thread crash kills the whole process; `fork/exec` interactions are subtle.

Cancellation

Asynchronous: cancelled at any instruction; unsafe (may leak locks/resources). **Deferred** (default): cancellation only at designated cancellation points (e.g., `pthread_cond_wait`).

Thread-safe vs reentrant

Thread-safe: works correctly when called concurrently (often via internal locks). **Reentrant**: works correctly when interrupted and re-entered by the same thread (e.g., from a signal handler) – requires no static state and no locks. Reentrant $\Rightarrow$ thread-safe; not vice versa.

fork/exec/exit semantics

fork duplicates only the calling thread (POSIX); other threads' state in the child may be inconsistent if they held locks. **exec** replaces the entire process image – all other threads cease to exist. **exit** terminates all threads in the process; `pthread_exit` terminates only the calling thread.

6 Synchronization

A **race condition** occurs when concurrent accesses to a shared resource yield non-deterministic outcomes due to interleaving (e.g., two threads doing `count++`). A **critical section** is code that accesses such shared state.

Critical-section requirements

- **Mutual exclusion**: at most one process inside at a time.
- **Progress**: if no one is inside, an entrant must eventually be selected.
- **Bounded waiting**: a finite bound on entries by others while a requester waits.

6.1 Software solution: Peterson (2-process)

```
flag[i] = true; turn = j;
while (flag[j] && turn == j) ; /* spin */
/* critical section */
flag[i] = false;
```

Satisfies all three requirements; assumes atomic single-word writes and no compiler reordering.

6.2 Threading atomics

- **Test-and-set** `TAS(&lock)`: atomically reads old value, sets to true, returns old. Spinlock: `while (TAS(&lock)) ;`.
- **Compare-and-swap** `CAS(&m, exp, new)`: if `*m==exp`, set to `new`, return true; else return false. Enables lock-free algorithms.

Spinlock vs blocking: spin is fine for very short critical sections; blocking (semaphore) is better when contention is long because spinning wastes CPU.

6.3 Semaphores

A **semaphore** is an integer with two atomic operations:

- **wait**(s) (P/down): if $s > 0$ then $s--$, else block until signal.
- **signal**(s) (V/up): $s++$ and wake one waiter.

Binary (saturates at 1; subsequent `signal` is a no-op) – equivalent to a mutex any thread can release. **Counting** (unbounded; general resources). `sem_init`, `sem_wait`, `sem_post`.

6.4 Monitors

A **monitor** is a high-level construct: only one process executes inside at a time. **Condition variables** `cv` support `wait(cv)` (release lock, sleep, re-acquire), `signal(cv)`, `broadcast(cv)`.

- **Hoare semantics**: signaled thread runs immediately; signaler resumes after.
- **Mesa semantics**: signaled thread becomes Ready; signaler keeps the lock. Predicate may have changed by wake-up – *must* re-check with `while`, not `if`. Reason: between signal and wakeup, another consumer can drain the buffer; on wakeup, with `if` the woken thread proceeds on a now-false predicate.

6.5 Classic problems

Bounded buffer (size N)

Semaphores `mutex=1`, `empty=N`, `full=0`.

#	producer	consumer
init	<code>mutex=1, empty=N, full=0</code>	
1	<code>wait(empty)</code>	<code>wait(full)</code>
2	<code>wait(mutex)</code>	<code>wait(mutex)</code>
3	<code>insert</code>	<code>remove</code>
4	<code>signal(mutex)</code>	<code>signal(mutex)</code>
5	<code>signal(full)</code>	<code>signal(empty)</code>

monitor form (Mesa):

```
pthread_mutex_lock(&m);
while (count == N)
    pthread_cond_wait(&notFull, &m);
buf[in] = x; in = (in+1)%N; count++;
pthread_cond_signal(&notEmpty);
pthread_mutex_unlock(&m);
```

Readers-writers

Multiple readers may hold simultaneously; writers exclusive.

- **Reader-preference**: readers count protected by `mutex`; first reader waits on `rw`, last signals. Writers may starve.
- **Writer-preference**: pending writer blocks new readers.

Reader-preference (semaphore form):

```
reader: wait(mutex); rc++; if(rc==1) wait(rw)
/*read*/; wait(mutex); rc--; if(rc==0)
writer: wait(rw); /*write*/; signal(rw);
```

Dining philosophers

n philosophers, n forks; each needs both adjacent forks. Naive symmetric solution **deadlocks**. Fixes: pick up both atomically; allow at most $n - 1$ to try; impose fork ordering (resource hierarchy); asymmetric (odd take left first, even take right first).

Priority inversion

A high-priority task is blocked on a lock held by a low-priority task; an unrelated medium-priority task preempts the low-priority holder, indirectly delaying the high-priority task. **Priority inheritance**: temporarily raise the holder's priority to the highest waiter's priority until it releases. **Priority ceiling**: every lock has a ceiling = max priority of any task that may use it; holder runs at the ceiling.

6.6 Deadlock

Four necessary conditions: **mutual exclusion**, **hold-and-wait**, **no preemption**, **circular wait**. Strategies:

- **Prevention**: negate one condition (e.g., total resource ordering breaks circular wait).
- **Avoidance**: Banker's algorithm – see below.
- **Detection & recovery**: cycle search in resource-allocation graph; abort or rollback.
- **Ostrich**: ignore (common when deadlock is rare).

Banker's algorithm

Maintain Available, Max, Allocation, Need=Max–Allocation. Loop: find i with $Need_i \leq Work$; $Work += Allocation_i$; mark i done; repeat. Safe iff every process gets marked. Example: Available

	Proc	Max	Alloc	Need
	P0	(7,5,3)	(0,1,0)	(7,4,3)
	P1	(3,2,2)	(2,0,0)	(1,2,2)
	P2	(9,0,2)	(3,0,2)	(6,0,0)
	P3	(2,2,2)	(2,1,1)	(0,1,1)
	P4	(4,3,3)	(0,0,2)	(4,3,1)

$P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$ (verify $Need \leq Work$ each step).

Resource-allocation graph

Bipartite directed graph: **request edge** $P \rightarrow R$, **assignment edge** $R \rightarrow P$. With single-instance resources, a *cycle* $\Leftrightarrow$ deadlock; with multi-instance resources, a cycle is necessary but not sufficient.

POSIX semaphore reference

`sem_t s`; `sem_init(&s, 0, 1)`; (last arg = initial count). `sem_wait(&s)` (decrement; block on 0); `sem_post(&s)` (increment; wake one). Named: `sem_open("/n", O_CREAT, 0644, 1)` – share between unrelated processes.

Lock-free vs wait-free

Lock-free: at least one thread always makes progress (no global stall) – usually a CAS retry loop. **Wait-free**: every thread completes in a bounded number of steps. Avoids deadlock and priority inversion at the cost of complex algorithms; needs strong memory barriers.

7 Memory Management

7.1 Address binding

Execution-time binding: MMU translates each access; enables page swapping (as compile/load-time which fix addresses earlier). **Logical address** is virtual address (what the CPU emits; **physical** is the actual RAM location. MMU performs the translation.

7.2 Base + limit registers
For contiguous schemes: `base` = start in physical memory, `limit` = size.
`phys = base + logical` if `logical < limit` else trap.
Hardware-enforced isolation between processes.

7.3 Contiguous allocation

Fixed partitions

Pre-allocated regions; **internal fragmentation** (allocated > used).

Variable partitions

Holes shrink as processes come and go; **external fragmentation** – total free $\geq$ request, yet no single hole large enough.

Placement

First-fit (first hole that fits; fast), **best-fit** (smallest hole; slowest scan, leaves slivers), **worst-fit** (largest hole; empirically poorest). First-fit is usually best in practice. **Next-fit**: like first-fit but resume scan from the position after the last allocation (roving pointer; not updated on `free`); avoids tiny-hole pile-up near the start.

Compaction

Slide allocated regions together; needs runtime relocation; expensive – temporary fix.

Buddy allocation

Power-of-two block sizes. Round request to next 2^k . Recursively split the smallest free 2^j (with $j > k$) into halves until size 2^k is reached. **Buddy rule**: blocks A , B of size 2^n are buddies iff $A \oplus B = 2^n$ (only bit n differs). On free, coalesce with buddy iff buddy is free and same size. **Internal frag** = 2^k – request. Allocation can fail when free fragments are not buddies (e.g., $32 + 64$ free $\nrightarrow$ 96 available).

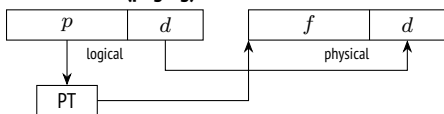
7.4 Paging

Logical and physical memory both partitioned into fixed-size blocks.

- Page** = unit of logical memory, size 2^k bytes; **frame** = same-sized unit of physical.
- Logical address = (p, d) : p = page number, d = offset (k bits).
- Per-process **page table** maps $p \rightarrow f$. Physical address = $f \cdot 2^k + d$.
- No external fragmentation**; only internal frag in last page (at most $2^k - 1$ bytes).
- PTE bits**: valid, protection (R/W/X), dirty, reference.
- Sharing**: multiple processes' PTEs can point to one frame (shared text, libc).

Page-table size pressure: 32-bit/4 KB $\Rightarrow 2^{20}$ entries per process; with 4 B per PTE that is 4 MB *per process*. 64-bit makes a flat table impossible – motivates multi-level / inverted (\$Virtual Memory).

Address translation (paging)



7.5 Segmentation

Logical address space is a set of **segments** (code, data, heap, stack, libraries) of *variable* size. Logical address = (s, d) . Per-process **segment table**: each entry holds (`base`, `limit`, protection).

$$\text{phys} = \text{base}_s + d \quad \text{if } d < \text{limit}_s.$$

- Per-segment protection**: code R+X, data R+W, stack R+W non-executable.
- Segments grow independently (e.g., heap up, stack down).

- External fragmentation** returns; no internal frag.

7.6 Paged segmentation

Combine: each segment is itself paged. The segment table entry points to a per-segment page table. Two-level translation: $(s, d) \rightarrow \text{segment table} \rightarrow \text{page table} \rightarrow \text{frame}, d \bmod 2^k$ stays as offset. Eliminates external fragmentation *and* keeps logical separation/protection.

Dynamic linking: shared library resolved at runtime via a stub; one frame shared across processes.

Swapping

Whole processes may be swapped to disk and back to free RAM (older systems). Modern OSes swap pages on demand (paging) instead.

Address translation worked

Small drill: 8-bit offset, 4-bit page number; logical $0x2BC \Rightarrow p = 0x2, d = 0xBC$. PT[$0x2$] = frame $0x7$; physical = $0x7BC$. Canonical 32-bit / 4 KB form: $d = 12$ bits (offset within page), $p = 20$ bits (1 M PTEs). VA = $0x12345678 \Rightarrow p = 0x12345, d = 0x678$.

Internal vs external fragmentation

Internal: wasted space *inside* an allocated unit (last page in paging; partition larger than process in fixed allocation). **External**: enough *total* free memory but no contiguous run satisfies the request (variable partitions, segmentation).

8 Virtual Memory

Virtual memory extends paging: a process's logical pages may live in RAM *or* on a backing store (disk). Per-PTE **valid bit** marks residency. Justified by **locality** (temporal + spatial): the cost of a fault is amortised over many subsequent local accesses.

8.1 Demand paging & page-fault flow

A page is brought in only on first reference. On access to page X :

- HW checks PTE; if valid, access RAM. Done.
- If invalid, trap to OS (**page fault**).
- OS locates X on disk; finds a free frame (or victimises one).
- Load X , update PTE valid+frame, restart faulted instruction.

Effective access time:

$$T_{\text{access}} = (1 - p)T_{\text{mem}} + pT_{\text{fault}}.$$

$T_{\text{fault}} \gg T_{\text{mem}}$, so p must be tiny. Goal of any replacement policy is to minimise p .

8.2 Page replacement

Evict a page when no free frame is available. **Dirty** pages must be written back; clean ones overwritten.

policy	eviction rule	Belady?	HW
FIFO	oldest in	yes	queue
OPT	farthest future use	no	oracle
LRU	longest unused past	no	timestamps/stack
Clock	ref-bit scan + 2nd chance	no	1 ref bit/PTE

LRU approximations

- Reference bit**: HW sets on access, OS clears periodically. Coarse "recently used" indicator.
- Second-chance / clock**: circular FIFO with a hand. If victim's ref bit is 0, evict; if 1, clear bit, advance, reconsider. Degenerates to FIFO when all bits are 1.
- Enhanced second-chance**: order victims by (`ref`, `dirty`) class – prefer (0, 0) $\rightarrow$ (0, 1) $\rightarrow$ (1, 0) $\rightarrow$ (1, 1) to avoid extra disk write-backs.

8.3 Frame allocation

- Equal**: N/M frames each.
- Proportional**: by process size (or priority): proc i gets $\frac{s_i}{\sum s_j} \cdot N$.
- Minimum frames**: arch-dependent lower bound to satisfy any single instruction.

Local replacement (only own frames): stable, but a starved process stalls itself. **Global** (any frame): self-adjusts but a hog penalises others.

8.4 Thrashing & working set

Thrashing: page-fault rate so high CPU is mostly idle servicing faults; cascades under global replacement. **Working set** $W(t, \Delta)$ = set of distinct pages referenced in the last Δ memory references; $WSS_i = |W_i|$. **No-thrashing condition**: $\sum_i WSS_i \leq m$ (total frames) $\Rightarrow$ no thrashing. Δ too small under-counts the locality; too large mixes two localities.

8.5 Page-table structures

Multi-level (hierarchical)

Split a flat 2^P -entry table into a top-level **page directory** and per-region inner tables. Address: [`dir`|`page`|`offset`]. Empty directory entries omit their inner tables – huge savings for sparse address spaces. Cost: extra memory access per level.

Inverted page table

One system-wide table with one entry per *physical* frame storing $\langle \text{pid}, p \rangle$. Size scales with RAM, not with virtual space. Lookup linear $\Rightarrow$ accelerated by hash on (pid, p).

8.6 TLB

A small associative hardware cache of recent PTEs. Hit ratio h . Generalised: $EAT = h(T_{\text{TLB}} + T_{\text{mem}}) + (1 - h)(T_{\text{TLB}} + (L + 1)T_{\text{mem}})$ for L -level paging. When code/data have different miss rates, compute ATT per stream and weight by access counts. **TLB vs faults**: TLB size affects translation cost (hit-rate), *not* page-fault rate; the latter depends on #frames + replacement policy.

COW after fork: see ch.2. **Memory-mapped files**: `mmap` attaches a file region into VA; reads/writes become memory accesses; pages demand-paged; dirty pages flushed by `msync` or process exit.

Aging (LRU approx)

Each page has an n -bit counter. On each clock tick: shift the counter right by 1, OR the reference bit into the high bit, clear the reference bit. Page with the smallest counter is the victim. Bounded state, no per-reference HW counter.

NRU (not-recently-used)

Class by (`ref`, `dirty`): 0=(0, 0), 1=(0, 1), 2=(1, 0), 3=(1, 1). OS clears `ref` bits periodically; on fault, evict any page from the lowest non-empty class.

TLB EAT (worked)

Let $T_{\text{TLB}} = 20$ ns, $T_{\text{mem}} = 100$ ns, $h = 0.95$, no faults. Two-level page table:

$$EAT = h(T_{\text{TLB}} + T_{\text{mem}}) + (1 - h)(T_{\text{TLB}} + 3T_{\text{mem}}) = 0.95 \cdot 120 + 0.05 \cdot 320 = 114 + 16 = 130 \text{ ns.}$$

Belady's anomaly (FIFO, ref string 1 2 3 4 1 2 5 1 2 3 4 5)

With 3 frames: 9 faults. With 4 frames: 10 faults. More frames $\rightarrow$ more faults – counter-intuitive; LRU and OPT do not exhibit this.

Multi-level PT footprint

For an L -level PT with PTE size s and inner table fitting one page (table entries determined by the per-level address split), let

n_ℓ = #pages used at level ℓ ($n_L = \#$ pages used; $n_{\ell-1} = \lceil n_\ell / (P/s) \rceil$). Footprint = $\sum_\ell n_\ell \cdot P$. **Worked** (3-level 10/8/6 split, 4 KB pages, 4 B PTE; level-3 has $2^6=64$ entries $\times$ 4 B = 256 B/table; level-2 has $2^8=256 \times$ 4 B = 1 KB/table; level-1 directory has $2^{10}=1024 \times$ 4 B = 4 KB; 256 KB process = 64 pages in one contiguous region): $n_3 = 1, n_2 = 1, n_1 = 1$. Footprint = $256 + 1024 + 4096 \approx 5.3$ KB.

Frame allocation worked

$N = 120$ frames, two procs $s_A = 200$ KB, $s_B = 400$ KB. Proportional: A gets $\frac{200}{600} \cdot 120 = 40$, B gets 80. Equal: 60 each.

Replacement worked

Reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 with 3 frames. Fault counts (full 20-ref string): FIFO 15, OPT 9, LRU 12. FIFO frame trace (first 10 refs; * marks faults):

ref	7	0	1	2	0	3	0	4
frames	7	70	701	012	012	123	230	304
F?	*	*	*	*	*	*	*	*

Page-fault EAT (slowdown)

Without page faults, $T = T_{\text{mem}}$. With fault rate p , $T_{\text{fault}} = 10$ ms, $T_{\text{mem}} = 100$ ns: $T = (1 - p) \cdot 100 + p \cdot 10^7$ ns. To keep slowdown under 10%: need $p < 10^{-6}$.

ASIDs (address space IDs)

Tag each TLB entry with a process ID; on context switch, no flush is needed – entries simply mismatch the new ASID and are bypassed. Avoids the cold-cache penalty after every switch.

9 File Systems

9.1 File abstraction

A **file** is a named sequence of bytes. **Metadata**: name, type, size, owner, permissions, timestamps (atime, mtime, ctime), block pointers. Operations: `open`, `read`, `write`, `close`, `lseek`, `stat`, `unlink`.

Key syscalls

- `int open(path, flags, mode)`: returns FD. Flags: `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREAT`, `O_APPEND`, `O_TRUNC`.
- `ssize_t read(fd, buf, n) / write(fd, buf, n)`: advance file offset.
- `off_t lseek(fd, off, whence)`: `SEEK_SET`, `SEEK_CUR`, `SEEK_END`.
- `int close(fd)`, `int unlink(path)`, `int stat(path, &buf)`.

Three-level FD indirection

Per-process FD table (in PCB; FDs 0/1/2 = `stdin/out/err`) $\rightarrow$ **system-wide open-file table** (per-open offset, mode, ref count) $\rightarrow$ **in-memory inode table**. Multiple opens of one file share an inode but get separate offsets; `dup/fork` share an open-file-table entry (and thus the offset). **OFT ref count** = #FDs pointing at the entry; `fork` increments by #parent FDs (each child FD shares); `close(fd)` decrements; entry freed at 0.

9.2 Directory structures

- Single-level**: one flat namespace; name collisions.
- Two-level**: per-user directory; isolated namespaces.
- Tree**: hierarchical; absolute (`/a/b`) vs relative (`./b`, `../b`); `.` and `..`.
- Acyclic graph**: hard/symbolic links; multiple paths to one inode.

Hard vs symbolic links

property	hard	symlink
indirection	0 (extra dirent)	extra inode
cross-FS	no	yes
can dangle	no	yes
target dirs	no (cycle risk)	yes
inc <code>link</code>	yes	no

9.3 File-system mount

`mount` attaches a FS at a directory; subsequent path resolution redirects at the mount point. Multiple FSes share one namespace.

9.4 Block allocation

Contiguous

All blocks consecutive. **+**: simple, fastest seek, random access via (start + i). **—**: external fragmentation; growth requires relocation.

Linked list

Each block stores a next-pointer. **+**: no external frag; trivial growth. **—**: random access is $O(n)$; pointer overhead inside data block; one corrupt link loses the rest.

FAT

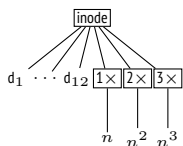
A separate **File Allocation Table** entry per disk block; the file's chain lives in the FAT, not the data blocks. **+**: random access via in-memory FAT; no in-block pointers. **—**: FAT is normally cached in RAM for fast random access; large volumes make this expensive. FAT corruption is catastrophic. Variants **FAT12/16/32** differ in entry width.

Indexed (inode)

Per-file index block lists data-block pointers. **+**: fast random access; no external frag. **—**: index-block overhead per file.

Multi-level inode (Unix ext2)

Inode holds 12 **direct** ptrs + 1 **single-** + 1 **double-** + 1 **triple-indirect**.



region	max file size ($B=4\text{KB}$, $s=4\text{B}$, $n=1024$)
direct only	$\leq 48\text{ KB}$
+ single	$\leq 4\text{ MB}+48\text{ KB}$
+ double	$\leq 4\text{ GB}$
+ triple	$\leq 4\text{ TB}$

Real-ext2 cap: 32-bit `i_size` bounds files at $\approx 2\text{ TB}$ regardless of fanout.

9.5 Free-space management

- Bitmap:** 1 bit per block; compact (n blocks $\rightarrow n/8$ bytes); fast bit-scan finds free runs.
- Linked list:** next-free-block pointer chain through free blocks; slow scan.
- Grouping:** a block holds pointers to next N free blocks plus a pointer to the next group block.
- Counting:** store (start, length) extents; compact when free regions are large.

9.6 Buffer cache

RAM cache of disk blocks. Reads check cache first; writes mark blocks dirty (**write-back**); a daemon flushes periodically; `sync/fsync/fdatasync` forces flush. Trades durability for throughput.

Directory implementation

Linear list (simple, $O(n)$ search), **hash table** ($O(1)$ avg, collisions), **B-tree** ($O(\log n)$).

9.7 Case studies

FAT

Boot sector $\rightarrow$ FAT1 $\rightarrow$ FAT2 (mirror) $\rightarrow$ root dir (fixed-size on FAT12/16) $\rightarrow$ data area. Directory entry: 8.3 name, attrs, first cluster, size. Cluster-level allocation unit.

FAT sizing

Max blocks = 2^w where w = usable entry-width bits (FAT16 reserves a few; usable $\approx 2^{16} - \text{reserved}$). Max partition = max-blocks $\times$ cluster size. Min pointer width = $\lceil \log_2(\text{total blocks}) \rceil$ rounded up to bytes. **On-demand FAT:** keep one FAT block resident; access to a non-resident entry costs +1 disk read.

ext2

Disk split into **block groups**, each with **superblock** (magic `0xEF53`, block/inode counts) (replicated), **group descriptor table**, **block bitmap**, **inode bitmap**, **inode table**, data blocks. 128-byte inode with 12+1+1+1 pointer scheme. Per-group locality reduces seeks.

Crash recovery

fsck: full-disk scan of bitmaps, inodes, directory hierarchy, link counts; slow on big disks. **Journaling** (ext3+, NTFS): write-ahead log of metadata changes commits atomically; replay on crash; fast recovery.

9.8 Disk scheduling

The disk arm has measurable seek time; the OS reorders pending I/O to reduce it.

- FCFS:** serve in arrival order. Fair, simple; poor seek locality.
- SSTF** (shortest-seek-time-first): pick the request closest to the current head. Lower mean seek; risk of **starvation** for distant requests.
- SCAN** (elevator): head sweeps end-to-end servicing requests in passing, then reverses.
- C-SCAN:** like SCAN but on reaching the end, jump back without serving in reverse — more uniform wait times.
- LOOK / C-LOOK:** as SCAN/C-SCAN but reverse at the last pending request, not the disk edge.

File protection (Unix): `rxwx` bits per owner/group/other (e.g., `0755`); `setuid` runs with the file owner's UID; `setgid` runs with the file's group GID; `sticky` bit on shared dirs (`/tmp`) restricts deletion to the file's owner.

File types (Unix)

Regular (data), **directory**, **symlink**, **character device** (unbuffered, byte-stream — `tty`, `/dev/null`), **block device** (buffered, sector — disks), **FIFO** (named pipe), **socket**.

Timestamps

atime (last access — costly to update on every read; many systems use `noatime`), **mtime** (last data write), **ctime** (last inode/metadata change).

Sparse files

A write past current EOF leaves a **hole**; no blocks allocated for the gap; reads return zeros. Saves space for large but mostly-empty

files (databases, virtual disks). `fcntl` explicitly allocates / punches holes.

Pathname resolution

Absolute (`/a/b/c`) starts at root inode; relative starts at CWD inode. For each component: read containing dir's inode, check execute permission, look up name, fetch target inode. Symlinks restart resolution at the target path (cycle detection needed). The kernel maintains a **dentry / DNLC** (directory-entry / name-lookup cache) to avoid repeated disk reads.

Permissions (octal)

Three triples (owner, group, other) of `rxwx`. Octal: $r = 4$, $w = 2$, $x = 1$. **0755** = `rwxr-xr-x` (typical executable); **0644** = `rw-r-r-` (typical file); **0700** = `rwX---` (private dir).

umask: default mode of new files = `mode & ~umask`; common `umask=022`.

Disk-scheduling example

Track 0.199, head at 53, queue 98 183 37 122 14 124 65 67.

- FCFS:** total head movement = $|53-98| + |98-183| + \dots = 640$.
- SSTF:** pick nearest each time $\Rightarrow 65, 67, 37, 14, 98, 122, 124, 183$; movement ≈ 236 .
- LOOK** (toward 0 first): 37, 14, 65, 67, 98, 122, 124, 183; ≈ 208 .
- C-SCAN** (toward 199, jump to 0): 65, 67, 98, 122, 124, 183, (199), (0), 14, 37; ≈ 382 .
- C-LOOK** (toward 183, jump to 14): 65, 67, 98, 122, 124, 183, 14, 37; ≈ 322 .

Block size trade-off

Block size: large = fewer ptrs / better seq throughput / more internal frag; small = opposite.

ext2 directory entry

Variable length record per file:

- `inode_no` (4 B), `rec_len` (2 B), `name_len` (1 B), `file_type` (1 B), then up-to-255-byte name.
- Deleted entries are coalesced into the previous entry's `rec_len` (form a hole) — fast unlink, slower lookup.

Journaling modes

writeback (metadata journaled only; data may be inconsistent on crash); **ordered** (default in ext3; journal metadata, write data before metadata commit); **full** (journal both data and metadata; safest, slowest).